



Context Is the New Security Boundary

Why enterprise agentic systems need runtime domain isolation, not just provider privacy promises.

A post-worthy working paper for enterprise AI leaders, architects, security teams, and builders.
Prepared for Henok Ghebrechristos - May 11, 2026

In enterprise AI, isolation cannot remain only a vendor promise. It has to become a runtime contract.

Runtime contract

Every artifact belongs to one domain. Every request runs in one active domain. Shared knowledge appears only through explicit publication - never silent global fallback.

The concern

The enterprise AI conversation is still too focused on one question: Will the provider train on my data? That question matters. But in agentic systems, it is no longer the deepest security question.

The deeper question is: What information is allowed to enter this reasoning episode?

OpenAI, Google Cloud, and other major providers now make serious enterprise commitments around data ownership, no-training defaults, tenant separation, encryption, retention, connector control, and administrative governance. These controls are important and should be acknowledged. But they solve only part of the problem. They mostly address provider-side data use and access control. They do not, by themselves, guarantee that an agent operating in one enterprise domain will not silently reason with context from another domain.

That distinction becomes critical as AI systems move from chat interfaces into enterprise workflows, internal knowledge, repositories, emails, spreadsheets, ticketing systems, SaaS tools, and long-running agents. The risk is not only data leakage in the old sense. The new risk is context bleed: information that was accessible somewhere becoming influential everywhere.

Post-ready thesis

No-training guarantees are not the same as runtime context guarantees. A provider can truthfully promise not to train on enterprise data while an agent still retrieves, injects, remembers, summarizes, or reasons across information that should not participate in the current task.

This is why the next security boundary is not only the database, the tenant, or the model-training pipeline. It is the active reasoning context.

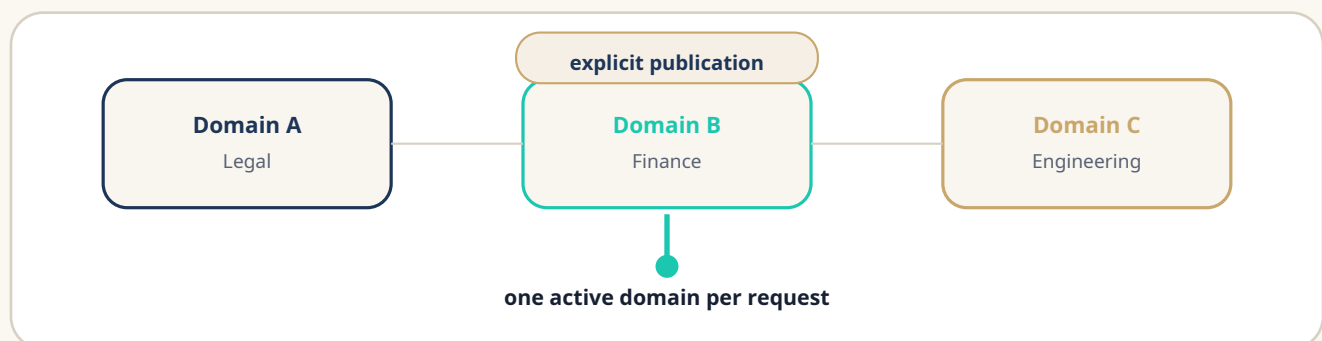


Figure 1. Domain isolation is not only about storage. It is about controlling which knowledge is eligible to participate in a specific request.

What providers already do well

The concern should be stated carefully. It is not a claim that enterprise AI providers are ignoring privacy. In fact, the official documents show meaningful controls.

OpenAI says enterprise and business customers own and control their business data, that OpenAI does not train its models on that data by default, and that organizations control connected internal sources and access features. Its API data controls also say API data is not used to train or improve models unless a customer explicitly opts in. [1][2]

Google Cloud similarly states that Gemini for Google Cloud does not use prompts or responses to train Gemini models, and Gemini Enterprise says customers own their data, that prompts, outputs, and training are not used to train Google models or models for other customers, and that customer data is not sold or used for advertising. [3][4]

Google Cloud's Gemini Enterprise Agent Platform is also moving toward runtime governance: Agent Platform includes Agent Identity, Agent Gateway, Model Armor, RAG Engine, Vector Search, session state, and persistent memory features. Google describes Agent Identity as a unique agent identity with least-privilege access and governance benefits, while Agent Gateway is positioned as a central policy enforcement point for tool calls and security policy. [5][6]

Provider/security layer	What it helps protect	What it does not fully settle
No-training default	Reduces the risk that enterprise prompts and outputs become model-training material.	Does not determine which internal sources may enter a specific request's context.
Tenant isolation and encryption	Protects storage and transmission boundaries across customers and services.	Does not define business-domain eligibility inside a reasoning episode.
IAM, RBAC, OAuth, SSO	Controls who or what can access systems and connectors.	A user may have broad access while a request should be narrowly scoped.
Agent identity and gateways	Give agents credentials, policy checkpoints, and auditable tool-call paths.	Still need explicit domain manifests, publication rules, and no global fallback semantics.

The fair evaluation

Provider privacy controls are necessary. They are not sufficient. They reduce vendor-side data risk, but they do not automatically impose domain-scoped reasoning inside the customer's enterprise workflow.

The deeper risk: semantic over-permissioning

Traditional security asks whether a subject can access an object. Agentic security must also ask whether a piece of knowledge is semantically allowed to influence the current task.

This is a different problem. A finance executive may legitimately have access to finance, strategy, HR, legal, procurement, and board materials. An enterprise assistant serving that executive may therefore have permission to retrieve from many systems. But a single request - for example, drafting a vendor negotiation email - should not automatically draw from unrelated HR investigations, acquisition discussions, legal strategy, or unreleased product plans. The user has access. The request should not inherit all of that access as active context.

That is semantic over-permissioning: the collapse of many authorized knowledge spaces into one undifferentiated reasoning space.

Context bleed

Context bleed occurs when information from one domain silently influences an output in another domain without explicit publication, request scope, policy approval, or provenance visible to the enterprise.

This risk becomes more serious in future agentic systems because the system is no longer merely answering questions. It may plan, compare, update artifacts, trigger tools, route tasks, create tickets, write code, draft emails, decide what evidence matters, and maintain memory. The more capable the agent becomes, the more important it is to know which knowledge was allowed to shape its reasoning.

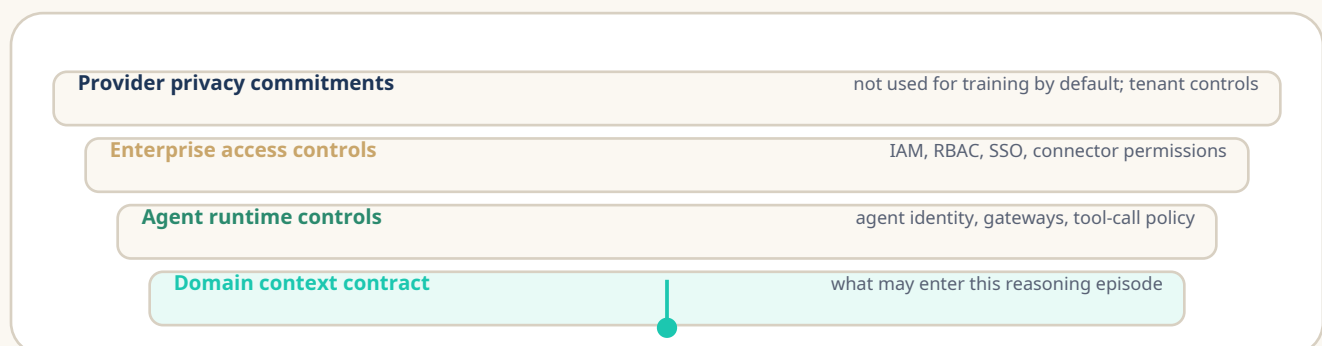


Figure 2. Provider privacy, enterprise access control, and agent runtime governance are necessary layers. The missing enterprise invariant is the active context boundary.

The simplest way to state the issue: access is not the same as relevance, relevance is not the same as authorization, and authorization is not the same as publication.

The runtime domain contract

The answer is not to reject provider platforms. The answer is to place a stricter enterprise contract above them: every reasoning act must occur inside a declared domain boundary.

Every artifact belongs somewhere. Every request runs somewhere. Nothing crosses domains silently.

A domain is not merely a folder or permission group. It is an enterprise knowledge boundary with a purpose, owner, policy, allowed artifacts, allowed tools, retrieval scope, logging requirements, publication rules, and escalation path.

A mature runtime contract should enforce these invariants:

- One active domain per request. The system must know the domain in which it is operating before retrieval, memory, tools, or generation begin.
- Artifacts are domain-owned. Documents, prompts, plans, code, decisions, summaries, and generated outputs should be tagged to a domain at creation time.
- Shared knowledge requires publication. Cross-domain knowledge should move through an explicit publish/share mechanism, not through hidden retrieval fallback.
- Retrieval is deny-by-default outside the active domain. The agent should not search globally unless the domain policy explicitly allows it.
- Memory is scoped. Agent memory should be partitioned by domain, with clear rules for what can be promoted to shared enterprise memory.
- Provenance is mandatory. The system should show what sources shaped an output, what domain they came from, and whether any cross-domain publication occurred.
- Exceptions are visible. Domain crossing should create an auditable event, not disappear inside an LLM trace.

The architectural move

Do not treat the LLM as the boundary. Treat the context assembler as the boundary. The strongest control point is the layer that decides what evidence, memory, tools, and artifacts are allowed into the prompt and action loop.

What enterprises should require

If this concern is taken seriously, enterprise AI governance should move beyond broad platform assurances and ask runtime-specific questions.

Question	Why it matters
Where did this answer get its context?	The enterprise needs to inspect the evidence boundary, not only the final output.
Which domain was active for the request?	A request without a domain is likely to become a global search over everything the user or agent can access.
Can the agent retrieve outside that domain?	Deny-by-default retrieval prevents accidental context bleed.
How does knowledge become shared?	Publication should be explicit, owned, versioned, and reversible.
Are memories scoped or global?	Persistent memory can become a hidden cross-domain channel unless carefully partitioned.
Can auditors reconstruct the context path?	Security teams need provenance, not just access logs.

This is also where provider tooling and enterprise architecture can meet. Agent identity, gateways, model armor, IAM, RBAC, connector scopes, RAG engines, vector search, and audit logs are useful building blocks. But the enterprise still needs to define the runtime policy: what is a domain, what belongs to it, what can be retrieved, what can be remembered, what can be published, and who approves boundary crossing.

The goal is not to slow down AI adoption. The goal is to make agentic AI safe enough to become infrastructure. Enterprises cannot build serious agentic systems on invisible context behavior. They need determinism around the boundary even when the model itself is probabilistic.

Demand this from agentic platforms

Show me the active domain. Show me the allowed context. Show me the denied context. Show me the provenance. Show me the publication path. Show me where global fallback is impossible by design.

Closing argument

Enterprise AI security is entering a new phase. The first phase was model access. The second phase was data privacy. The third phase will be context governance.

In that third phase, the central question will not be only whether the provider trained on the data. It will be whether the enterprise can prove which data was allowed to shape the answer, which domain owned the artifact, which memory was consulted, which tool was called, and which boundary was crossed.

That is why the runtime domain contract matters. It turns a vague trust problem into an enforceable system invariant. It says that enterprise agents do not float in a global sea of company knowledge. They operate inside declared domains. They publish what must be shared. They cite what shaped their output. They deny silent fallback.

The enterprise of the future will not be secured only by preventing data from leaving. It will be secured by controlling how knowledge is allowed to participate in reasoning.

References

- [1] OpenAI. Enterprise privacy at OpenAI. Updated January 8, 2026. <https://openai.com/enterprise-privacy/>
- [2] OpenAI. Data controls in the OpenAI platform. <https://developers.openai.com/api/docs/guides/your-data>
- [3] Google Cloud. How Gemini for Google Cloud uses your data. Last updated May 8, 2026. <https://docs.cloud.google.com/gemini/docs/discover/data-governance>
- [4] Google Cloud. Gemini Enterprise app: Best of Google AI for Business. <https://cloud.google.com/gemini-enterprise>
- [5] Google Cloud. Agent Platform overview - Gemini Enterprise Agent Platform. <https://docs.cloud.google.com/gemini-enterprise-agent-platform/overview>
- [6] Google Cloud. Agent Identity overview - Gemini Enterprise Agent Platform. <https://docs.cloud.google.com/gemini-enterprise-agent-platform/govern/agent-identity-overview>